

# Phase 10B - README

---

- [PHASE 10B — AI Explanation Layer](#)
  - [Real Estate AI Decision Intelligence Platform](#)
  - [What was built](#)
  - [The core mechanism](#)
  - [Honest disclosure](#)
  - [Interactive chat CLI](#)
  - [How to run](#)

## PHASE 10B — AI Explanation Layer

---

### Real Estate AI Decision Intelligence Platform

---

The real LLM-based explanation layer (\$10.10) — completing all 10 phases of the Master A-Z Prompt. Reads a Phase 9 `DecisionResult` and explains it in business language, with a hard, code-level guarantee that the LLM cannot alter any protected number or the decision itself.

### What was built

---

| File                                                       | Purpose                                                                                                                                                                                                                                                           |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/explanation/ai_explanation_layer.py</code>       | The real Gemini API integration + safe fallback logic                                                                                                                                                                                                             |
| <code>src/explanation/numeric_fidelity_validator.py</code> | Deterministic governance check — verifies every protected number (IRR, NPV, cap rate, ROI, DSCR, decision score, decision confidence, risk probabilities, location score, market strength) is reproduced correctly before an LLM response is ever shown to a user |
| <code>src/explanation/test_phase10b.py</code>              | 15 automated tests (all passing, includes 1 post-delivery audit-fix regression test)                                                                                                                                                                              |
| <code>examples/</code>                                     | 3 real fallback examples (INVEST/AVOID/INSUFFICIENT) + 1 clearly-labeled simulated LLM response demonstrating the validator end-to-end                                                                                                                            |

Full detail: `docs/phase10b_report.md` , `docs/limitations.md` .

### The core mechanism

---

1. Phase 9's `orchestrator.run_decision_pipeline()` produces a `DecisionResult` — entirely deterministic, no LLM involved.
2. `ai_explanation_layer.generate_ai_explanation()` sends that `DecisionResult` to Gemini with a strict system prompt: explain it, don't change any number, don't override the decision.
3. `numeric_fidelity_validator.validate_fidelity()` extracts every protected number directly from the `DecisionResult` and checks the LLM's text reproduced each one it mentions correctly.
4. **If validation fails — a number is altered, or the decision state is changed — the LLM's response is discarded entirely** and Phase 9's deterministic template is returned instead, with a clear, logged reason.

### Honest disclosure

---

No `GEMINI_API_KEY` was available in this build sandbox — same situation as “no live PostgreSQL” throughout this platform. The no-key fallback path was exercised for real; the LLM-response-handling path was tested with mocked responses (both faithful and deliberately tampered), disclosed directly in the test

file. Two real bugs were found in the validator itself through this adversarial testing — see [docs/phase10b\\_report.md](#) Section 5. (This build was also migrated from Anthropic to Gemini and had an inherited Phase 6 ROI bug fixed during a post-delivery audit — see Section 5B.)

## Interactive chat CLI

`chat.py` (at the project root) is a terminal Q&A interface over the whole platform — ask it anything, not just “explain property X”:

```
python3 chat.py
```

```
Ask a question: top 5 properties
Ask a question: safest investment
Ask a question: compare 2731 and 5053
Ask a question: locality Shanker Gardens
Ask a question: why were some ML models rejected?
Ask a question: 2731
```

Every question is routed through `src/chat/query_router.py` into one of five handled paths (single property / compare / locality / ranking / general question) — see [docs/phase10b\\_report.md](#) Section 5C for the full design rationale, including a real bug this rebuild fixed: a naive earlier draft matched any bare 4-digit number as a property id (misfiring on years/percentages), returned “top N” results in arbitrary CSV row order rather than sorted by any real metric, and had no path at all for general questions. This version runs the real decision pipeline to rank properties, resolves localities by name (not just numeric id), and grounds general questions in the platform’s own documented facts — 29/29 new automated tests in `test_chat.py`, plus the original 15/15 in `src/explanation/test_phase10b.py` (no regressions).

## How to run

```
cd phase10b
pip install -r requirements-phase10b.txt --break-system-packages

# 1. Set up your environment file (never commit the real .env)
cp .env.example .env
# then edit .env and fill in DB_USER / DB_PASSWORD / GEMINI_API_KEY

# Without a Gemini API key set -- uses Phase 9's deterministic template
python3 -c "
import sys; sys.path.insert(0, 'src/agents'); sys.path.insert(0, 'src/explanation')
from orchestrator import run_decision_pipeline
from ai_explanation_layer import generate_ai_explanation
out = run_decision_pipeline(2731)
result = generate_ai_explanation(out['decision_result'])
print(result['source'], '--', result['fallback_reason'])
print(result['explanation'])
"

# With GEMINI_API_KEY set in .env -- makes the actual Gemini call,
# validated before being returned. No `export` needed -- .env is loaded
# automatically by python-dotenv (see db/connection.py and
# src/explanation/ai_explanation_layer.py).
python3 -c "... same as above, generate_ai_explanation will now call the real API ..."

# Tests
python3 -m pytest src/explanation/test_phase10b.py -q
```

## Environment variables ( `.env` )

This project reads all credentials from a `.env` file rather than hardcoding them anywhere. `.env.example` documents every variable used across the platform (DB connection for the live-database path, Gemini API

key/model for the explanation layer). Setup:

```
cp .env.example .env
# edit .env with a text editor and fill in real values
```

`.env` is listed in `.gitignore` and must never be committed — only `.env.example` (which contains no real secrets) should be shared or checked into version control. `db/connection.py` and `src/explanation/ai_explanation_layer.py` both call `load_dotenv()` at import time, so once `.env` exists, no `export` command is needed — just run the Python scripts directly and the variables are already in `os.environ`. Real environment variables (e.g. set by a deployment platform) always take priority over anything in `.env`.